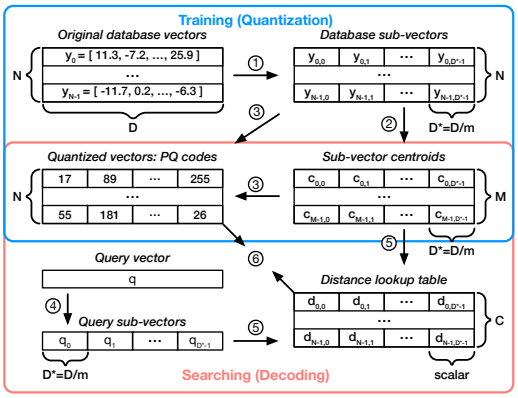

Figure 2: Product quantization (PQ) for vector search.

$D^* = \frac{D}{m}$, typically ranging from 4 to 16 in practice. A clustering algorithm is performed in each sub-space ② to obtain a list of centroids c, allowing each database sub-vector to be approximated by its nearest centroid. Typically, the number of clusters per sub-space is set as $M = 256$, such that a cluster ID can be represented with one byte. Thus, once the cluster centroids are stored, each database vector can be represented by m-byte PQ codes.

Searching (decoding). A query vector is compared against the quantized database vectors. The distance computation can be formulated as $\hat{d}(x, y) = d(x, c(y)) = \sum_{i=1}^{m} d(x_i, c_i(y_i))$, where $\hat{d}(x, y)$ is the approximate distance between a query vector x and a quantized database vector y, and $c(y)$ is the reconstructed database vector using the PQ codes and the cluster centroid vectors per sub-space. To calculate $\hat{d}(x, y)$, the query vector is divided into m sub-vectors (x_i) ④ and compared against the reconstructed quantized sub-database-vectors $c_i(y_i)$. To speed up distance computations given many database vectors, a distance lookup table ⑤ can be constructed and reused within a query, encompassing all combinations between a sub-query-vector and a cluster centroid within the same sub-space. With this table, the value of $d(x_i, c_i(y_i))$ can be swiftly retrieved by looking up the table with the PQ code as the address ⑥, leading to improved computational efficiency.

2.3 Motivation: Efficient RALM Inference

An efficient RALM inference engine should meet the following **system requirements**:

- Both the LLM inference and the large-scale vector search components should be fast and resource-efficient.
- The system should be flexible enough to accommodate diverse RALM configurations, spanning various combinations model sizes, database sizes, and retrieval frequencies.

However, little effort has been devoted to developing efficient RALM systems that meet the above requirements. This is likely because RALM has been an emerging topic within the machine learning community [7, 31, 32, 42, 49], with their prototype implementations exhibiting the following problems:

(P1) Each research RALM system focuses on *being able to run* one or a small number of RALM models, paying little attention to latency, throughput, resource efficiency, and system flexibility.

(P2) While hardware accelerators for LLMs, such as GPUs, are advancing rapidly, less attention has been paid to the vector search aspect, which, as our evaluations will demonstrate, can become the performance bottleneck in RALM inference.

(P2.1) CPUs are slow in scanning PQ codes during query time ⑥. This is due to the frequent cache accesses (for each byte of PQ code, load the code and use it as an address to load a distance) and the instruction dependencies between operations (distance lookups depend on PQ codes and distance accumulations depend on the lookup values). Even with the state-of-the-art SIMD-optimized CPU implementation [1], the throughput peaks at roughly 1 GB/s per core when scanning PQ codes (1.2 GB/s on Intel Xeon Platinum 8259CL @ 2.50GHz). Within a CPU-memory-balanced server, the PQ code scanning process significantly underutilizes the available memory bandwidth, as about 16 cores are required to saturate the bandwidth of a single memory channel (around 20 GB/s).

(P2.2) GPUs suffer from two major limitations for large-scale vector search. Firstly, the limited memory capacity of each GPU makes large-scale searches on GPU clusters cost-prohibitive. For instance, accommodating only 1 TB of PQ codes necessitates at least 16 NVIDIA A100 GPUs (cost 300K USD as of March 2024), each with 80 GB of memory, given that a portion of memory should be reserved for intermediate search states. Although an alternative solution is to adopt a hybrid CPU-GPU architecture where the GPU fetches vectors from CPU's memory, the inter-processor bandwidth is way lower than the GPU memory bandwidth. Even for NVIDIA Grace Hopper, with the latest high-performance CPU-GPU interconnect, the single-direction bandwidth of 450 GB/s is only 15% of the GPU's bandwidth. Secondly, the throughput for PQ code scanning on GPUs is considerably lower than the GPU's bandwidth, only around 50% of the bandwidth even with large batch sizes (evaluated on NVIDIA A100), due to the multiple passes of memory accesses to write and read intermediate results at each search step [40].

3 CHAMELEON: SYSTEM OVERVIEW

We design and implement Chameleon, an efficient, flexible, and performant RALM inference system:

- Chameleon employs heterogeneous hardware to accelerate both LLM inference and vector search efficiently.
- Chameleon disaggregates the accelerators, enabling independent scaling for each type of hardware, thus supporting various RALM configurations efficiently.
- The modular design of Chameleon allows flexible hardware upgrades, such as integrating more powerful LLM inference accelerators or ASIC-based ChamVS accelerators in the future.

Figure 3 overviews the Chameleon architecture, which primarily consists of the following components.

Firstly, ChamVS is a distributed accelerator engine for low-latency vector search. On the one hand, ChamVS.idx is a GPU-based IVF index scanner colocated with the ChamLM GPUs (right side of Figure 3). While Chameleon also supports index scan on CPUs, GPUs are generally more favorable for handling this embarrassingly parallel workload due to their superior memory bandwidth and computational capability. Given that GPUs are already integrated into Chameleon, no additional devices are required. The only overhead is a slight increase in GPU memory usage, as the index sizes are small relative to the database vectors. For example,

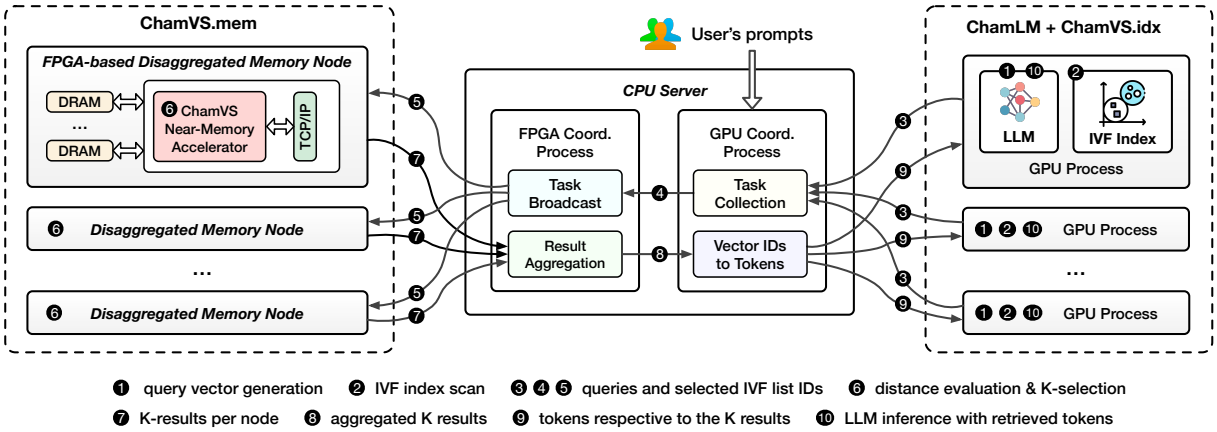

Figure 3: Chameleon is a heterogeneous and disaggregated accelerator system for efficient RALM inference.

assuming 1KB per vector and one thousand vectors per IVF list, a single GB of index can support one million IVF lists, enough for a large database containing one billion vectors. On the other hand, ChamVS.mem is responsible for querying quantized database vectors. ChamVS.mem contains one or multiple disaggregated memory nodes, each with a partition of the database vectors and a near-memory retrieval accelerator prototyped on FPGA for query processing (left side of Figure 3).

Secondly, ChamLM is a multi-GPU LLM inference engine, as shown on the right side of Figure 3. Each GPU, managed by an independent GPU process, can reside on the same or different servers. Currently, ChamLM assigns each GPU a full copy of the LLM, as RALMs can achieve high generation quality even with smaller LLMs [7, 49]. Future larger models could be accommodated by extending ChamLM to support tensor or pipeline parallelism [59, 69, 73] across GPUs. Once a retrieval request is sent, a GPU pauses inference to wait for results. While one potential solution to avoid such GPU idleness is to split the generation into two sub-batches — one executes inference when the other one is waiting for retrieved contents — this approach does not necessarily improve performance. This is because (a) using sub-batches reduces inference throughput, and (b) retrieval latency may not align with inference latency.

Thirdly, the CPU serves as the cluster coordinator, managing the lightweight communication between the GPUs and FPGAs. After receiving search requests from the GPU processes, it dispatches them to the FPGA-based disaggregated memory nodes, aggregates the per-partition results returned by the FPGAs, converts the K nearest neighbor vector IDs into their corresponding texts, and sends the retrieved tokens back to the GPUs. Since each query only requires less than ten KBs of network data transfer, the communication latency is negligible compared to vector search and LLM inference.

Token generation workflow. For each token generation step, the procedure diverges depending on whether the retrieval is invoked. Without retrieval, the GPUs infer the next token as in regular LLMs. With retrieval, the first step is to generate a contextual query vector ❶, either by using the hidden state of the current context [41, 42] or encoding the query tokens through another model [7]. Following this, the IVF index residing on the same GPU is scanned to select the *nprobe* most relevant IVF lists ❷. The query

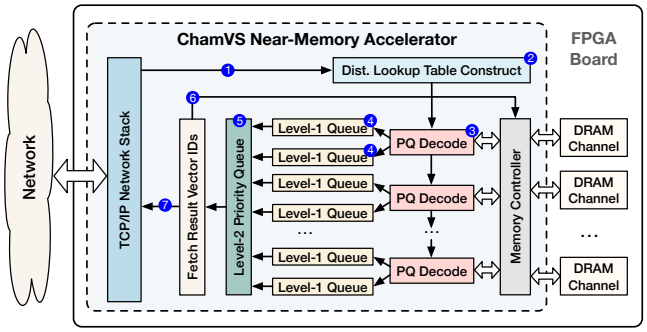

Figure 4: The ChamVS near-memory retrieval accelerator.

vector and the list IDs are then transmitted to the GPU coordinator process running on the CPU node via the network ❸. After recording the association between queries and GPU IDs, the query and list IDs are forwarded to the FPGA coordination process ❹, which broadcasts them to the FPGA-based disaggregated memory nodes ❺. The ChamVS near-memory processor on each node then uses the query vectors to construct distance lookup tables for each IVF list, computes the distances between the query and quantized database vectors, and collects the K nearest neighbors ❻. Subsequently, the result vector IDs and distances from all memory nodes are sent back to the CPU server ❼, which aggregates the results ❽ and returns the tokens of the nearest neighbors to the originating GPU ❾. Finally, the GPU predicts the next token based on both the context and the retrieved tokens ❿.

4 CHAMVS NEAR-MEMORY ACCELERATOR

ChamVS enables high-performance, large-scale vector search by pairing each disaggregated memory node with a near-memory retrieval accelerator. As shown in Figure 4, the accelerator comprises a distance lookup table construction unit, several PQ decoding units for distance evaluations between query vectors and quantized database vectors, a group of systolic priority queues for parallel K-selection, and multiple memory channels.

4.1 PQ Decoding Units

As shown in Figure 4 ❸, each ChamVS accelerator contains multiple PQ decoding units to fully utilize the memory bandwidth. These